

INF1103 Project Part 1 Report

Practical Group 4

Team 8

Project Title:

AI-Powered Student Mental Well-being Assessment System

Done By:

Team Members	Student ID	Email
Kaung Khant Min Thwin	2602036	2602036@sit.singaporetech.edu.sg
Tay Brenden Lomibao @ Tay Yi San (Leader)	2601795	2601795@sit.singaporetech.edu.sg
Ning Wee Jung	2605753	2605753@sit.singaporetech.edu.sg
Mohamed Azfar S/O Abdul Raheem	2605812	2605812@sit.singaporetech.edu.sg
Marcus Wong Yu Xuan	2603182	2603182@sit.singaporetech.edu.sg

Date of submission: 14 October 2026

Introduction

University students face increasing academic demands, tight deadlines, and limited time for revision, which can contribute to stress, anxiety, and burnout. However, many struggle to recognise or express their emotions before their mental well-being begins to affect their academic performance and quality of life. Although existing wellness applications offer mental health resources and general coping strategies, they may not provide sufficiently personalised insights based on individual students' emotional states, sleep patterns, and academic workload. To address this gap, our team developed an AI-powered mental wellness application that analyses students' inputs to identify potential warning signs and provide tailored recommendations to support their well-being.

Data Structure and Storage Design

Files and Data Formats

The application uses two JSON files, users.json to store user account information and records.json to store mental well-being assessment records. We chose JSON over CSV because it supports key-value pairs and nested data structures, allowing related information to be organised within a single record.

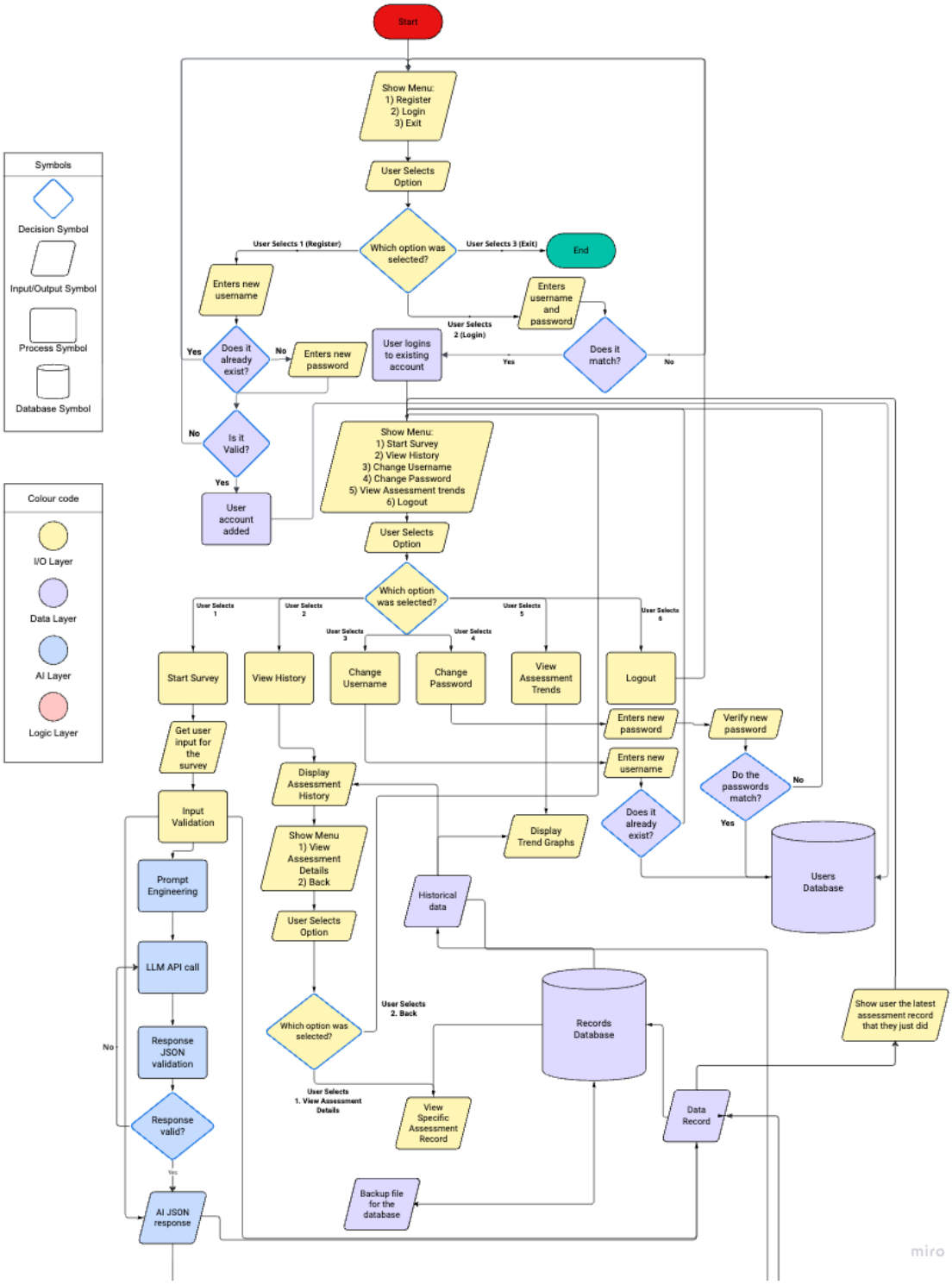
Structure of stored records

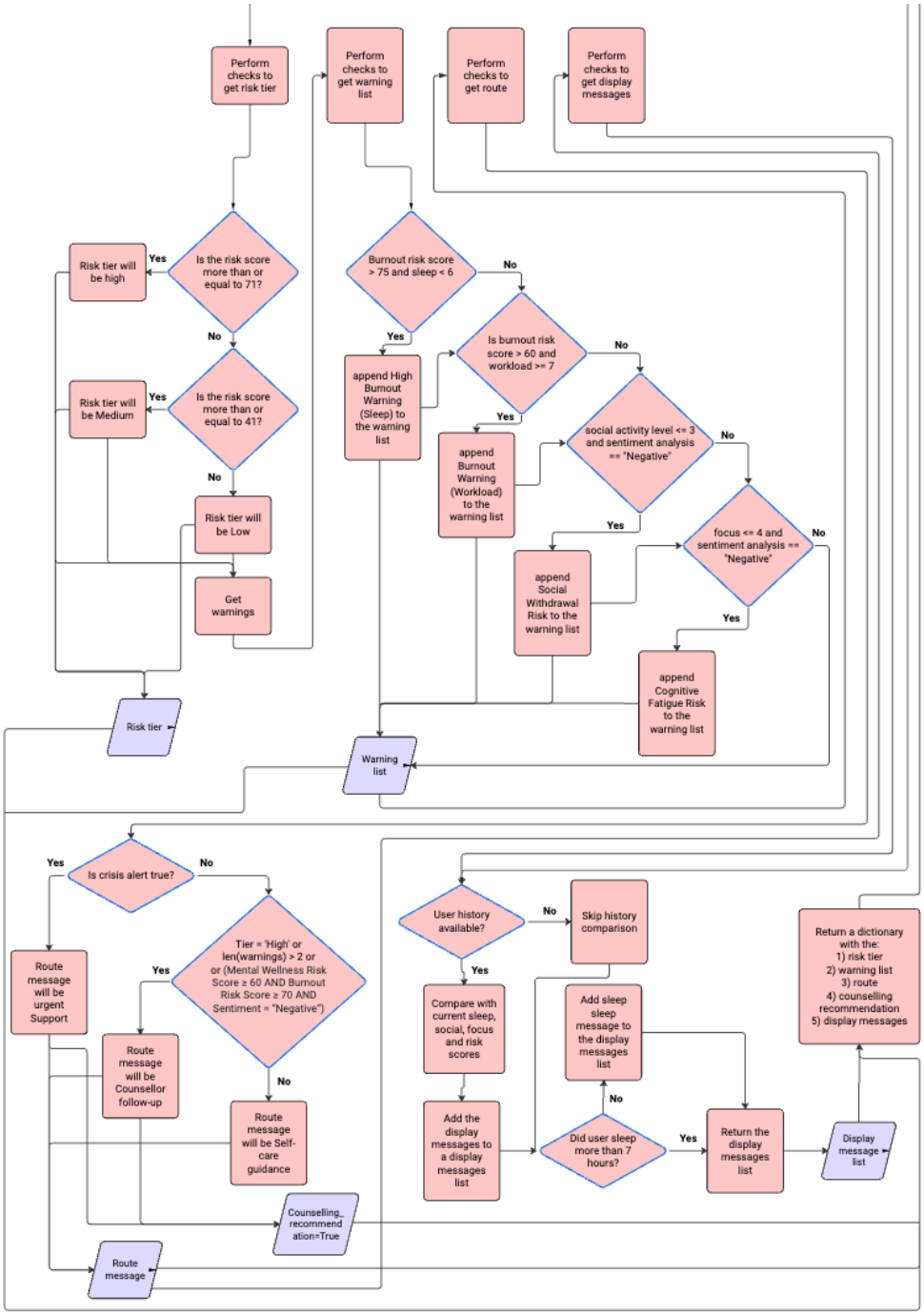
The users.json file stores user accounts as a list of dictionaries, where each element represents one user and contains a unique user_id, username and hashed password.

The records.json file stores assessment records as a list of dictionaries, where each element represents one assessment. Each record contains a unique record_id, the assessment date and time, and three nested dictionaries: user_input, ai_output and logic_output. These store the user's responses, AI-generated analysis and recommendations, and the results of the predefined business rules, respectively. The following is a sample JSON schema illustrating the structure of an assessment record.

```
[{
  "record_id": "",
  "date": "",
  "time": "",
  "user_input": {},
  "ai_output": {},
  "logic_output": {}
}]
```

Data Flow Diagram




Perform checks to get risk tier
Perform checks to get warning list
Perform checks to get route
Perform checks to get display messages
Is the risk score more than or equal to 71?
Yes
Risk tier will be high
No
Is the risk score more than or equal to 41?
Yes
Risk tier will be Medium
No
Risk tier will be Low
Get warnings
Risk tier
Burnout risk score > 75 and sleep < 6
Yes
append High Burnout Warning (Sleep) to the warning list
No
Is burnout risk score > 60 and workload >= 7
Yes
append Burnout Warning (Workload) to the warning list
No
social activity level <= 3 and sentiment analysis == "Negative"
Yes
append Social Withdrawal Risk to the warning list
No
focus <= 4 and sentiment analysis == "Negative"
Yes
append Cognitive Fatigue Risk to the warning list
Warning list
Is crisis alert true?
Yes
Route message will be urgent Support
No
Tier = 'High' or len(warnings) > 2 or or (Mental Wellness Risk Score ≥ 60 AND Burnout Risk Score ≥ 70 AND Sentiment = "Negative")
Yes
Route message will be Counsellor follow-up
No
Route message will be Self-care guidance
Counselling_ recommend ation=True
Route message
User history available?
No
Skip history comparison
Yes
Compare with current sleep, social, focus and risk scores
Add the display messages to a display messages list
Add sleep sleep message to the display messages list
No
Did user sleep more than 7 hours?
Yes
Return the display messages list
Display message list
Return a dictionary with the:
1) risk tier
2) warning list
3) route
4) counselling recommendation
5) display messages

miro

Exception Handling Matrix

Exception Scenario	Possible cause	Detection Method	Handling Strategy	User Outcome
Network connection error	The user is not connected to the internet while using the program.	The API function throws an APIConnectionError exception.	Use a try-except code block to catch the error and output the relevant information to the user.	The user is notified to connect to the internet.
API key error	Mishandling of the API key in the Google AI Studio dashboard.	The API function raises a ValueError exception.	A backup API key is used.	The backup API key restores AI functionality for the user.
AI model unavailable	Updates to the Gemini API pertaining to the renaming/removal of models.	The API function returns an invalid status code.	A backup model is used.	The backup model restores AI functionality to the program.
Database file does not exist	Records.json has not been created yet.	The load function checks whether the file exists.	Creates a new JSON file containing an empty list.	The application can start with an empty database.
Invalid JSON format in database	File contains corrupted or incorrectly formatted JSON.	The load function catches a json.JSONDecodeError exception when reading the file.	Use a try-except code block to catch the error and returns an empty list. The latest backup file can be used to restore the database.	The user is notified that the database failed to load.
Invalid record data type during saving	The record contains data that cannot be serialised to JSON.	The save function catches TypeError during the saving process.	Use a try-except code block to catch the error.	The user is notified that the record failed to save in the database.
No previous assessments available	The user is completing their first survey, or has no records before today.	Comparison functions check if not previous_attempts.	Return an empty message list and skip historical calculations.	The user receives the current assessment without historical comparison messages.